

HOSTED BY



FIGHT NIGHT

BLUE CORNER

DEVELOPER

STOP **FIGHTING** YOUR
SPRING BOOT TESTS

TUE 22 SEP 2026 • 2 PM UTC+2

Software Craft Community @ DATEV

VS

RED CORNER

SPRING BOOT
TEST SUITE

Referee: Philip Riecks

Achieve Confidence in Every Commit





Stop **Fighting** Your Spring Boot Tests

Why your tests fight back - and how to make them fight for you 🥊

Software Craft Community @ DATEV · September 22, 2026

About Philip

- Self-employed developer from [Herzogenaurach](#), living in Erlangen, Germany (close to Nuremberg) 🍺
- Met Sven Hansen during the [Spring I/O](#) conference in Barcelona this year 🇪🇸
- Blogging & content creation with a focus on [testing Java Spring Boot](#) applications 🌱
- Founded [PragmaTech GmbH](#) - We Help Developers to Frequently Deliver Software with More Confidence 🚤



Participate During the Talk

Go to menti.com and use the code **7735 0833** to **anonymously** submit answers:



Please answer the **first three questions**:

- Who wrote your tests last week?
- How long does your test suite take end to end?
- How confident are you deploying on a Friday afternoon (0 to 10)?

Why Test Software?



In 2026, most of our code is **not
written by us.**



Frog told the LLM to write tests for all the code it wrote.

“There,” he said. “Now the model can’t write broken or buggy code.”

“But the model can write broken or buggy tests,” said Toad.

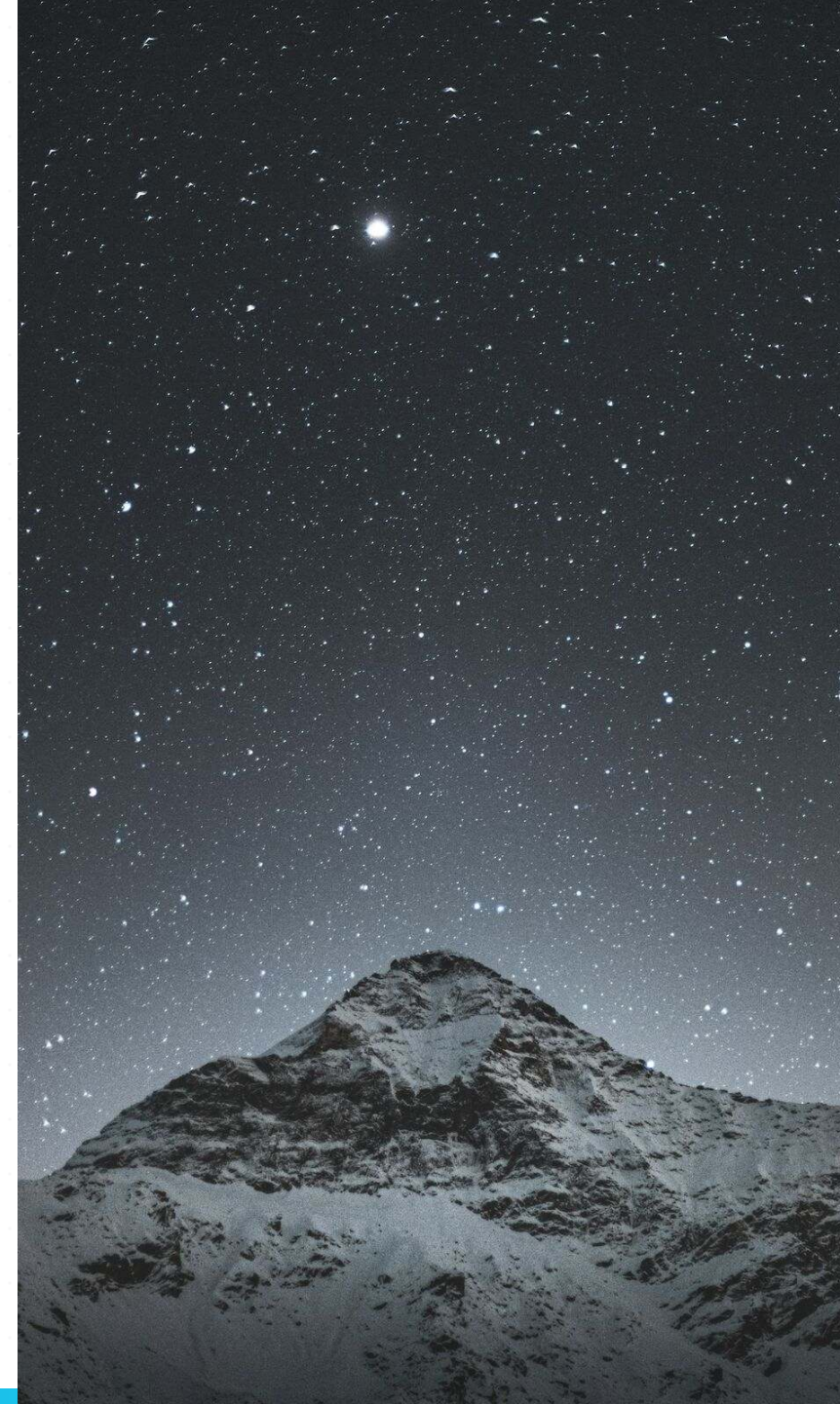
“That is true,” said Frog.

My Overall Northstar for Engineering Excellence

Imagine seeing this pull request on a Friday afternoon:



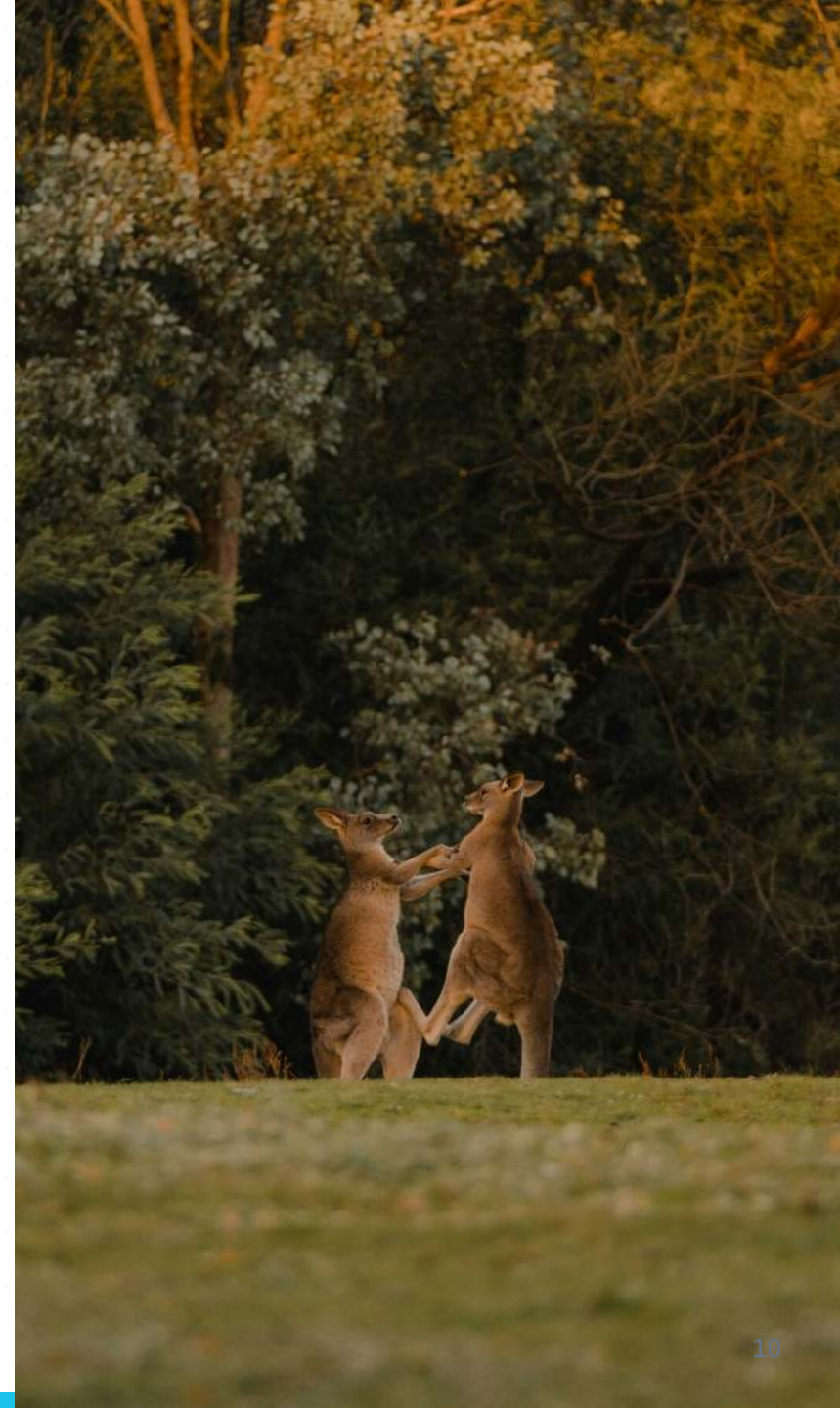
How confident are you to merge this major Spring Boot upgrade and deploy it to production once the pipeline turns green?



**Good tests don't just catch bugs -
they give you [fast feedback](#) and
[confident deployments](#).**

When Your **Spring Boot Tests** Fight Back

- The suite takes > 30 minutes, so nobody runs it before pushing
- A test goes red and you cannot tell whether the code or the test is broken
- You reach for `@SpringBootTest` because it is the only thing that works
- The pipeline is green and you are hesitant to deploy



The Three Spring Boot Testing Myths We Bust Today

01 "I need @SpringBootTest for that."

02 "Spring Boot tests are slow."

03 "It is green, so it works."

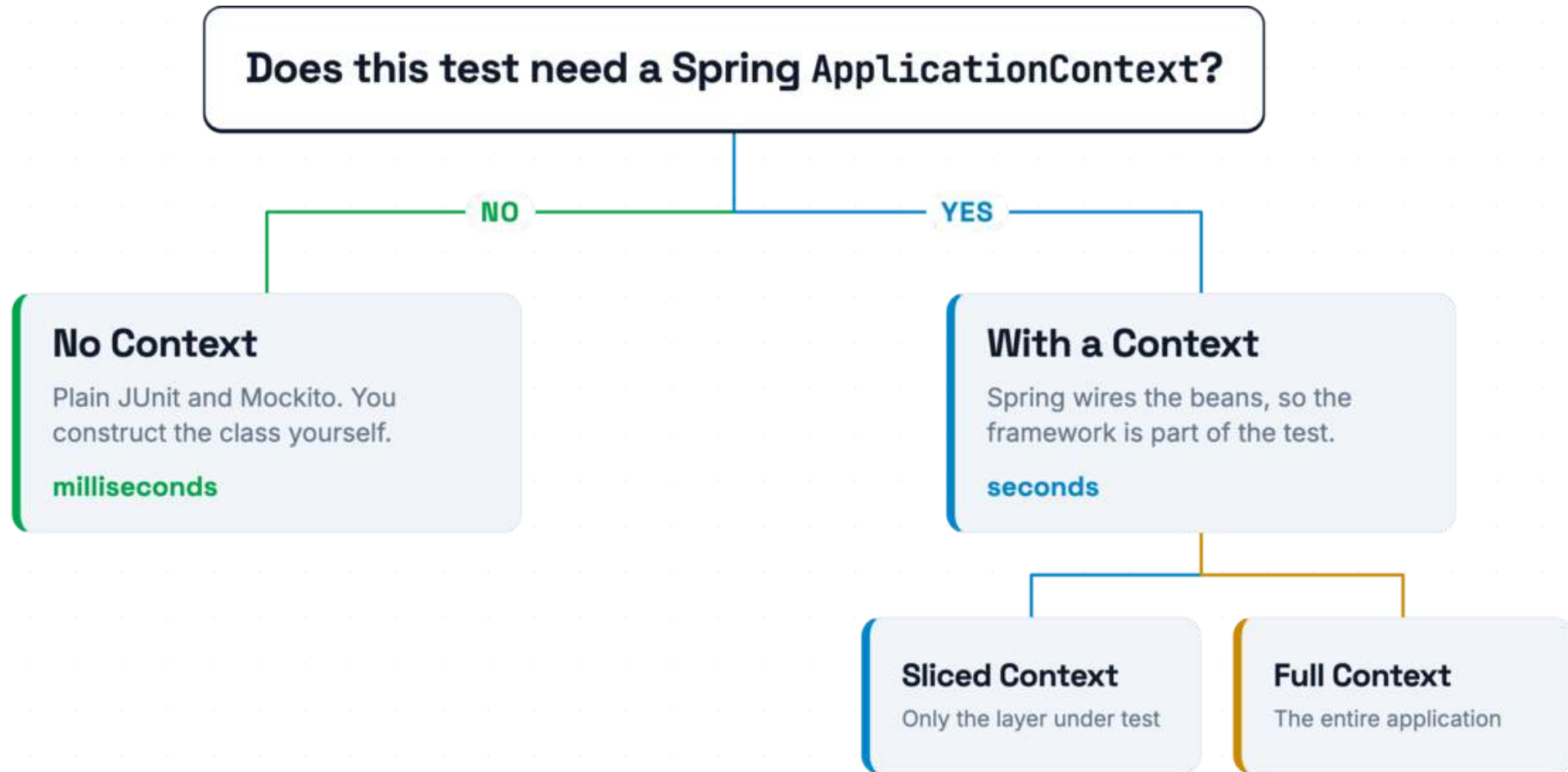
MYTH #1

“Every test needs the full context.”

Spoiler: you don't always need `@SpringBootTest`.



Three Ways to Write Tests for Spring Boot applications



Do We Even Need a Context?

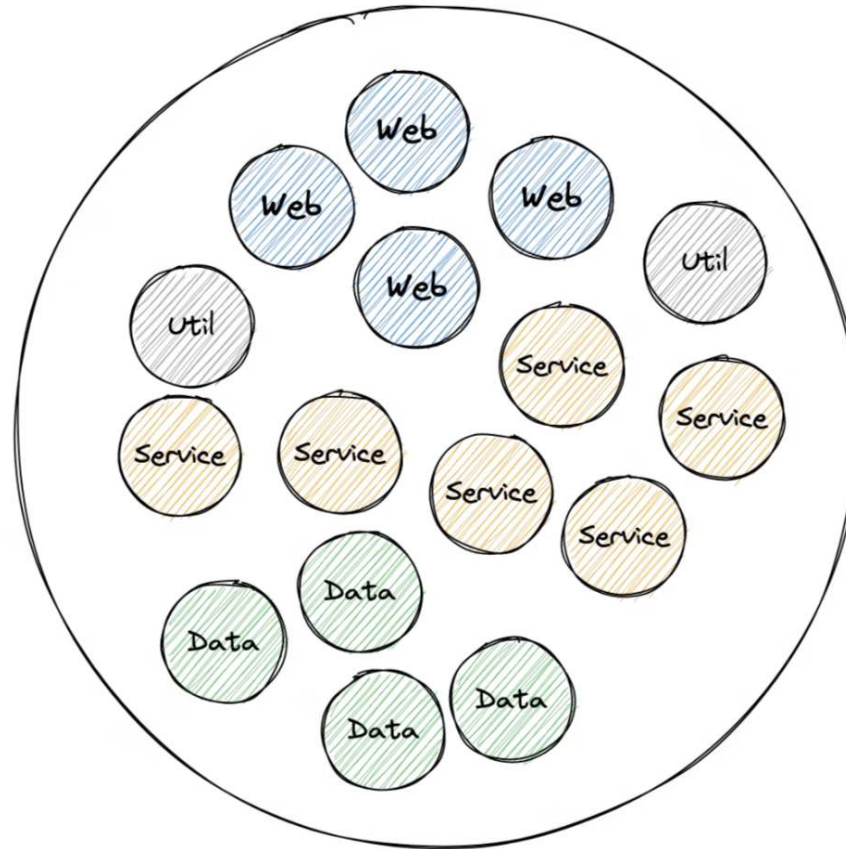
A plain unit test is fine

- Business logic: calculations, branching, state transitions
- Validation rules you wrote yourself
- Verify small units of work independently
- Usually the core parts of your application

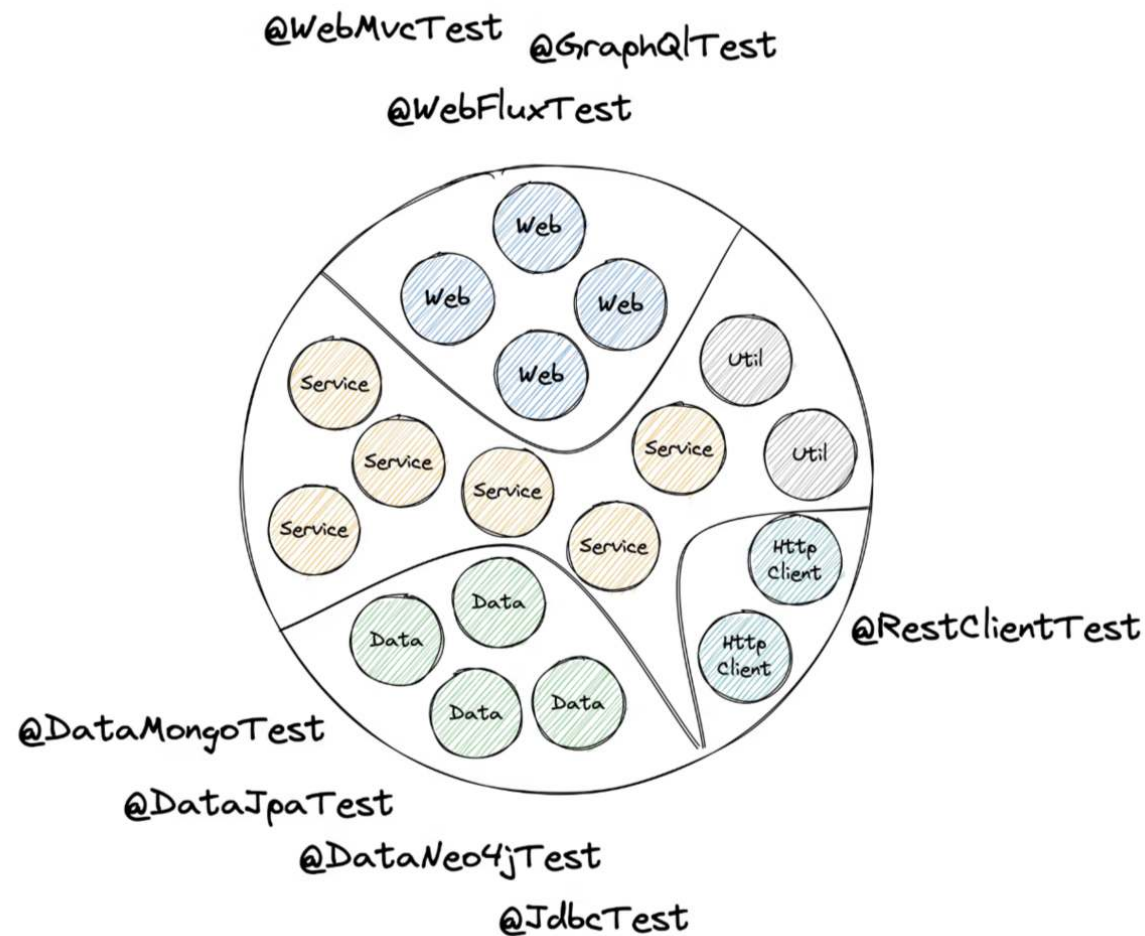
But it can never tell you

- Does `@PreAuthorize` actually **block** that caller?
- Does `GET /api/customers/{id}` **map** to this method?
- Does that JPQL return what you think against **real SQL**?
- Does an invalid body give **400**, not an accidental 201?

A Typical Spring `ApplicationContext`



You Don't Always Need the Entire Context



Sliced or Full Context?

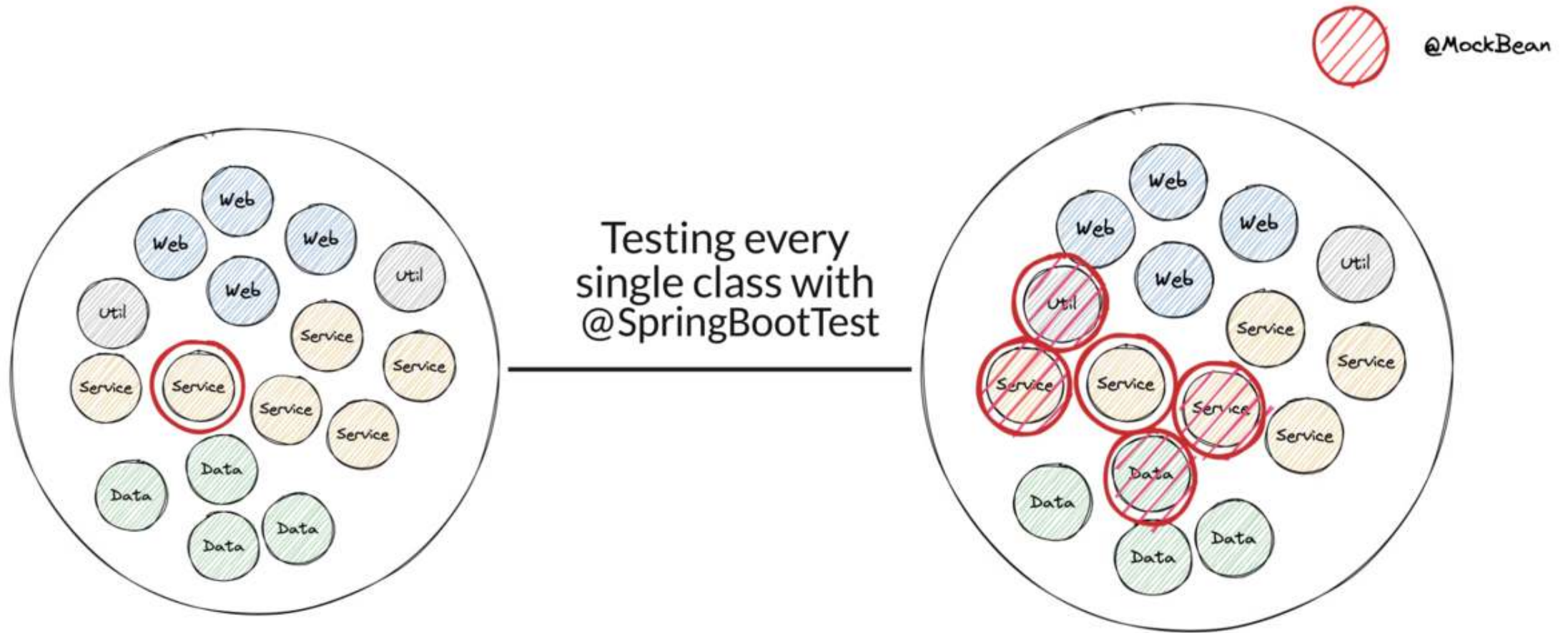
Slice it when

- You verify **one layer's contract** with Spring
- The collaborators behind it can be **replaced** with `@MockitoBean`
- You want framework behaviour **without paying** for the whole application
- Annotations of choice: `@WebMvcTest`, `@DataJpaTest`, `@JsonTest`, etc.

Take the full context when

- One flow **crosses several layers** end to end
- You need everything wired **as in production**
- Verification of entire user journeys
- Annotation of choice: `@SpringBootTest`

The `@SpringBootTest` Obsession



Decision Paralysis: When to Include a Context

A simplified decision table:

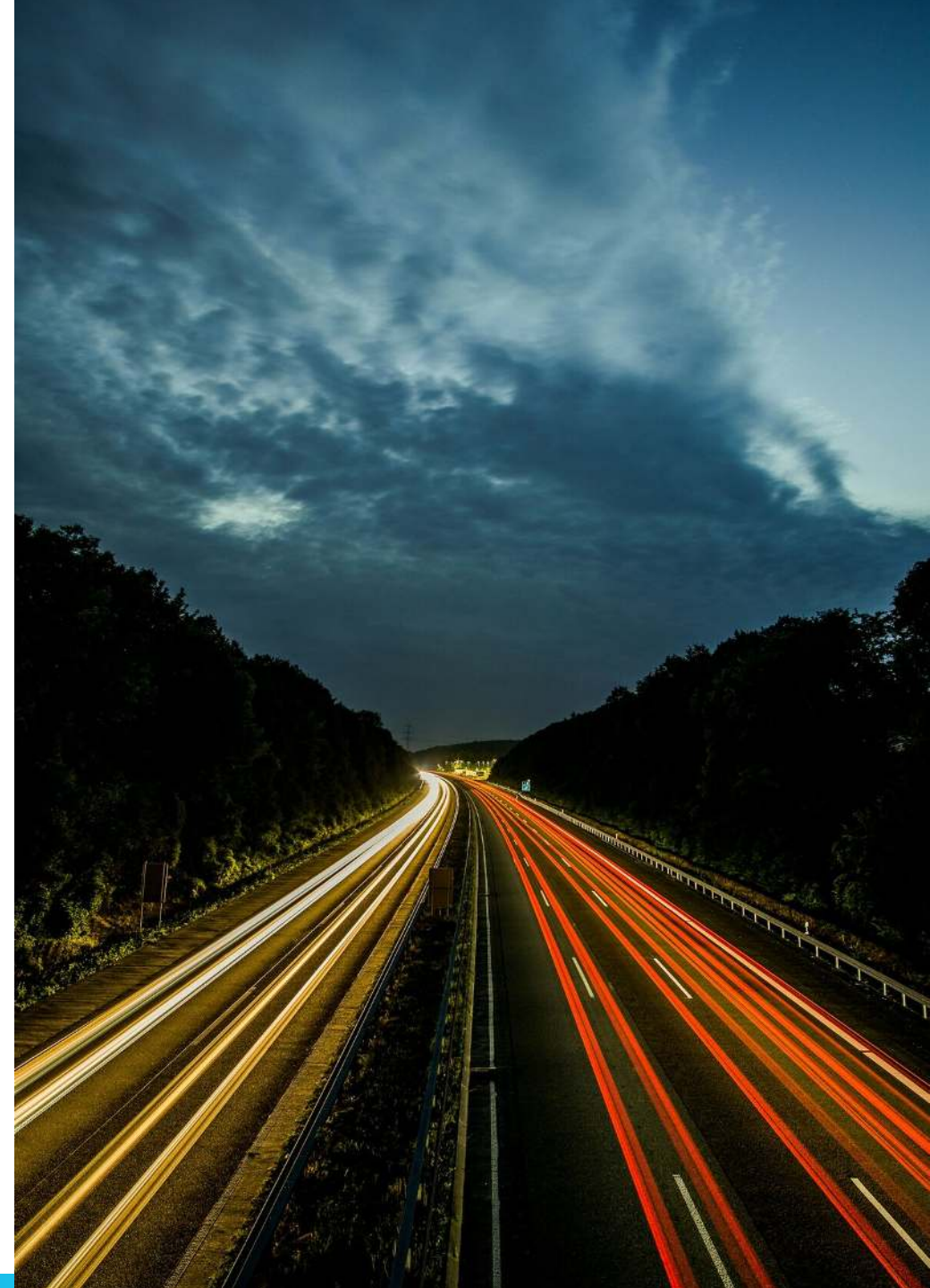
Question	Tool
Does my business logic work?	Plain JUnit + Mockito, no Spring
Does my HTTP layer map, validate, serialize?	<code>@WebMvcTest</code>
Does my query return what I think?	<code>@DataJpaTest</code>
Does my client talk to the remote API?	<code>@RestClientTest</code> (including a mock HTTP server)
Does the whole thing start and work end to end?	<code>@SpringBootTest</code>

Strategy: Pick the test type that gives you the **most confidence in your safety net for the **cheapest execution time**.**

MYTH #2

“Spring Boot tests are slow.”

You are not paying for tests. You are paying for context starts.



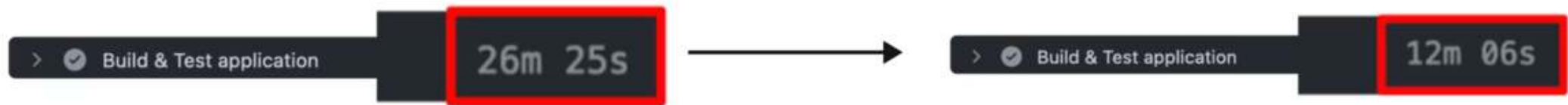
Slow Tests Are Not the Problem. Slow **Feedback** Is.

Attack it from four angles:

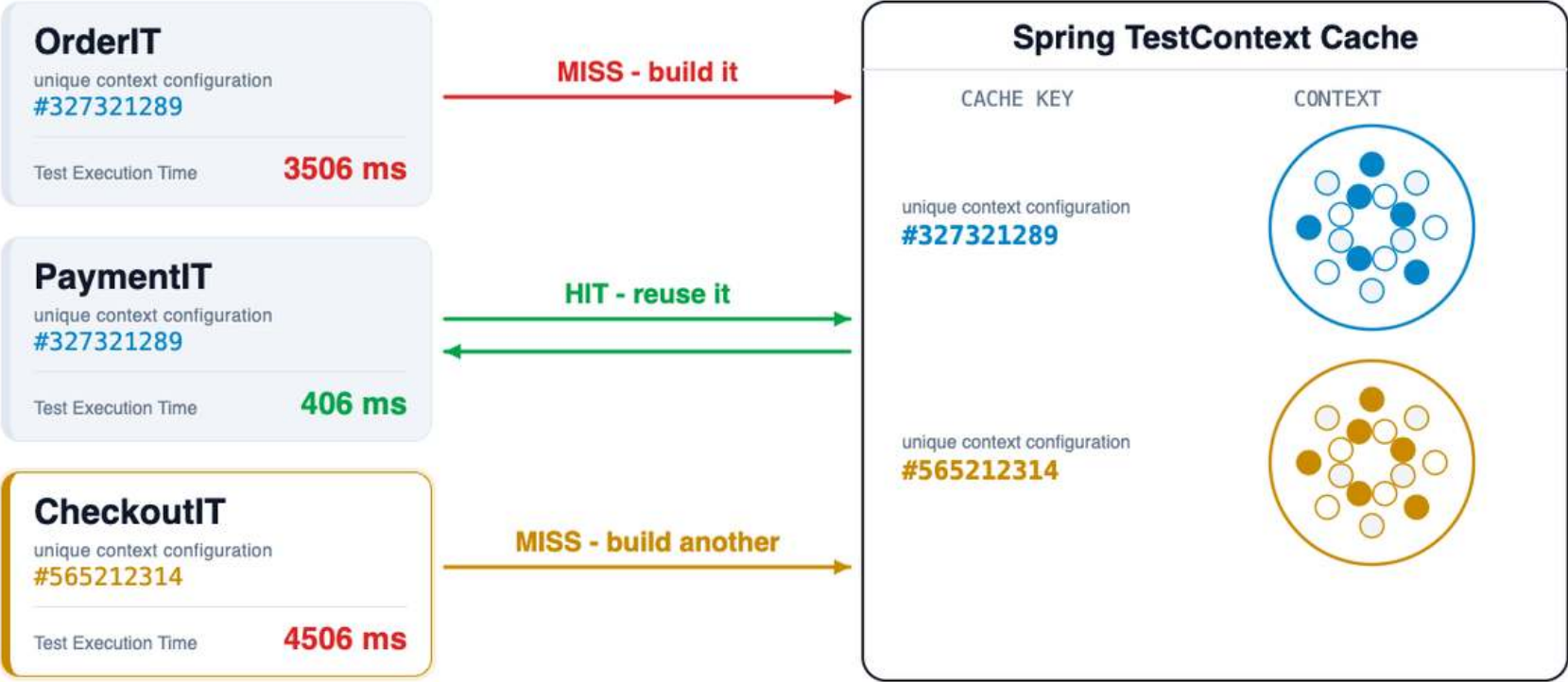
1. **Right test level** - the cheapest test that answers the question (Myth #1)
2. **Fewer context starts** - Spring Test's hidden gem
3. **Parallel execution** - use the cores you are paying for

Angle 2: Spring Context Caching (Spring Test's Hidden Gem)

- **The problem:** integration tests need a started and initialized `ApplicationContext`, which takes time
- **The solution:** the Spring TestContext framework caches a started context for later reuse
- Part of every Spring Boot project already, via `spring-boot-starter-test`
- Speed improvement example:



Context Caching in Action



Result: 3 tests ran, but 2 contexts had to start

Only **PaymentIT** reused a cached context and finished fast - the other two each paid for a fresh one.

How the Cache Key is Built

```
1 // DefaultContextCache.java
2 private final Map<MergedContextConfiguration, ApplicationContext> contextMap =
3     Collections.synchronizedMap(new LruCache(32, 0.75f));
```

This goes into the cache key (`MergedContextConfiguration`):

- activeProfiles (`@ActiveProfiles`)
- propertySourceProperties (`@TestPropertySource`)
- contextCustomizer (`@MockitoBean` , `@MockBean` , `@DynamicPropertySource` , ...)
- etc.

Every unique combination is a new context start.

Detect Context Restarts - Visually

The screenshot shows an IDE window with a test results panel on the left and a console output panel on the right. The test results panel shows two tests passing: `OrderControllerReusableTest` and `CustomerControllerReusableTest`, both with a duration of 233 ms. The console output shows the following log messages:

```
2022-05-09 11:24:33.653 DEBUG --- [main] o.s.test.context.BootstrapUtils :  
Instantiating CacheAwareContextLoaderDelegate from class [org.springframework.test.context.cache  
.DefaultCacheAwareContextLoaderDelegate]  
2022-05-09 11:24:33.662 DEBUG --- [main] o.s.test.context.BootstrapUtils :  
Instantiating BootstrapContext using constructor [public org.springframework.test.context  
.support.DefaultBootstrapContext(java.lang.Class,org.springframework.test.context  
.CacheAwareContextLoaderDelegate)]  
2022-05-09 11:24:33.687 DEBUG --- [main] o.s.test.context.BootstrapUtils :  
Instantiating TestContextBootstrapper for test class [de.rieckpil.talks.introduction  
web
```

Below the log messages, there is a red box containing a Spring Boot logo and the text:

```
:: Spring Boot :: (v2.6.7)
```

At the bottom of the console, there is another log message:

```
2022-05-09 11:25:51.412 INFO 43090 --- [main] d.r.t.introduction.OrderControllerTest  
: Starting OrderControllerTest using Java 17 on Philips-MacBook-Pro.local with PID 43090  
(started by rieckpil in /Users/rieckpil/Development/git/talks/fixing-a-broken-window)
```

Detect Context Restarts - with Logs

```
<logger name="org.springframework.test.context" level="TRACE" />
```

```
2022-05-05 14:27:38.136 DEBUG 42500 --- [          main] org.springframework.test.context.cache : Spring test
ApplicationContext cache statistics: [DefaultContextCache@68e62b3b size = 1, maxSize = 32, parentContextCount = 0,
hitCount = 11, missCount = 1]
2022-05-05 14:27:38.136 DEBUG 42500 --- [          main] tractDirtiesContextTestExecutionListener : After test class:
context [DefaultTestContext@325bb9a6 testClass = ApplicationIT, testInstance = [null], testMethod = [null], testException
= [null], mergedContextConfiguration = [WebMergedContextConfiguration@1d12b024 testClass = ApplicationIT, locations =
'{}', classes = '{class de.rieckpil.talks.Application}', contextInitializerClasses = '[]', activeProfiles = '{}',
propertySourceLocations = '{}', propertySourceProperties = '{org.springframework.boot.test.context
.SpringBootTestContextBootstrapper=true, server.port=0}', contextCustomizers = set[org.springframework.boot.test.context
.filter.ExcludeFilterContextCustomizer@5215cd9a, org.springframework.boot.test.json
.DuplicateJsonObjectContextCustomizerFactory$DuplicateJsonObjectContextCustomizer@9257031, org.springframework.boot.test
```

Detect Context Restarts - with Tooling



An [open-source Spring Test utility](#) that provides visualization and insights for Spring Test execution, with a focus on Spring context caching statistics.

Overall goal: Identify optimization opportunities in your Spring Test suite to speed up your builds and ship to production faster and with more confidence.

Angle 3: Test Parallelization

Goal: use the cores you already pay for.

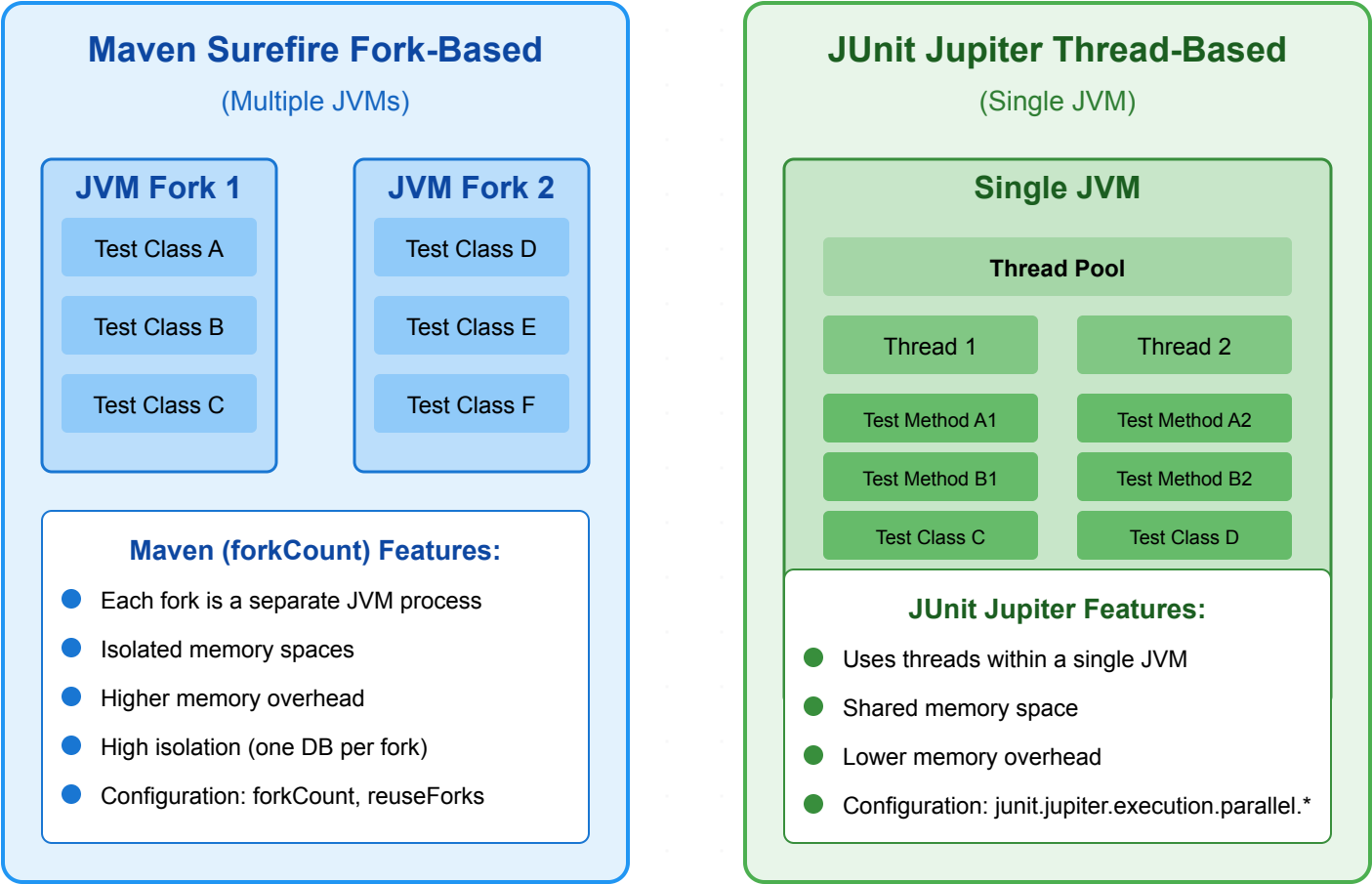
Requirements:

- No shared state
- No dependency between tests or their execution order
- No mutation of global state

Two ways to get there:

- Fork a new JVM with Maven/Gradle - more resources, fully isolated
- Use JUnit Jupiter's parallel execution mode - same JVM, multiple threads

Java Test Parallelization Options



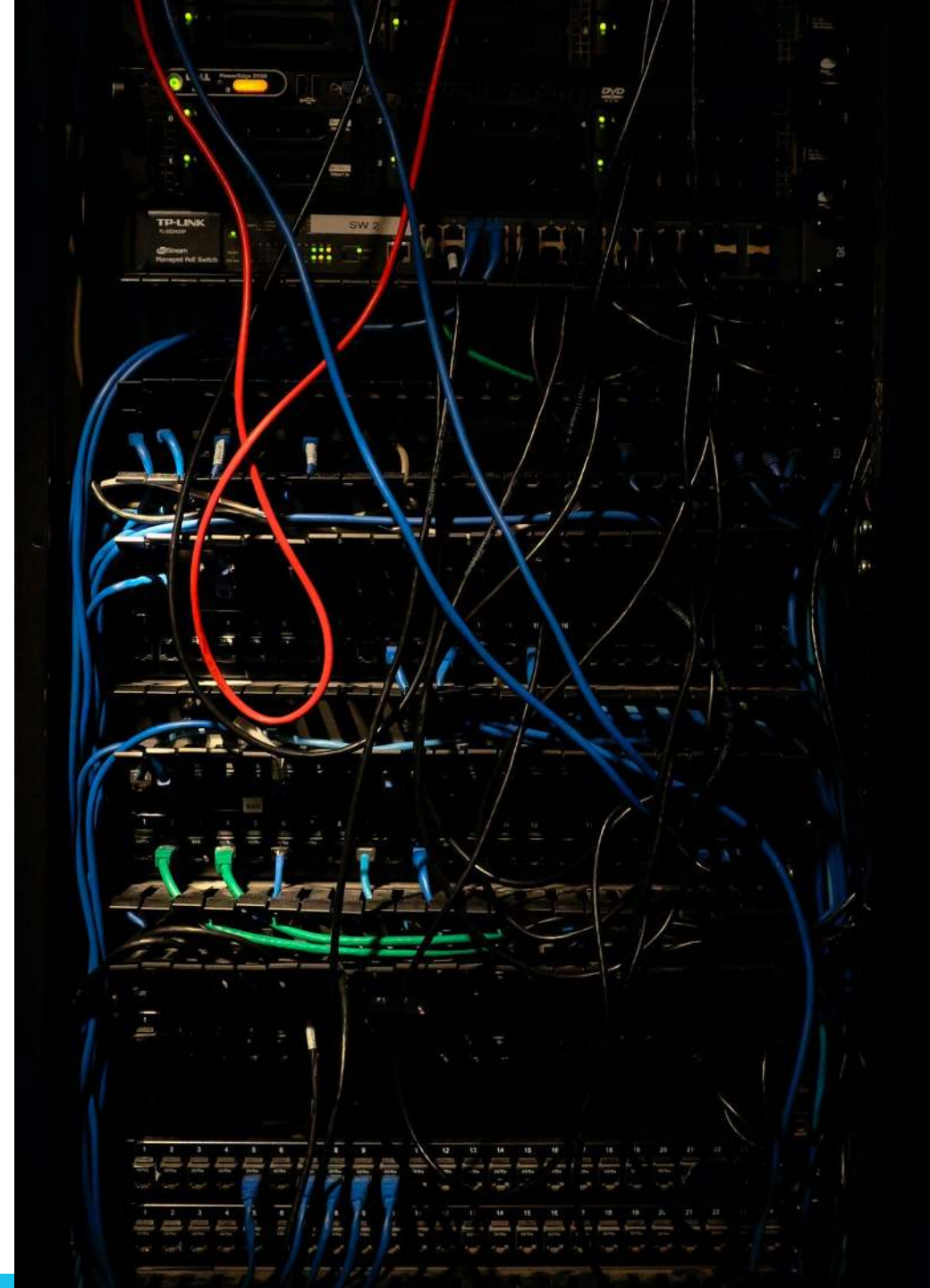
Make the Most of Both

- Understand how the cache key is built before you add another annotation
- Avoid `@DirtyContext`, especially in central base classes
- Monitor and investigate context restarts instead of guessing
- Align the number of unique context configurations across the suite
- Turn on parallel test execution, start selectively

MYTH #3

“It is green, so it works.”

Green proves your tests ran. Not that they would have caught anything.



Where Tests Drift Away From Production

- **The database:** H2 in tests, PostgreSQL in production. Different SQL dialect, different constraints, different behaviour.
- **Remote services:** mocked away with Mockito, so the HTTP layer, the serialization and the error handling are never exercised.
- **The test itself:** it asserts that the code ran, not that the code is right.

Each gap is a place where a green build still ships a bug.

Must-Have #1: Testcontainers (Real Infrastructure)

Use case: your test needs the real database, broker or cache - not an in-memory stand-in.

```
1 @Container // <-- Testcontainers manages the lifecycle of the container
2 @ServiceConnection // <-- automatically configures Spring Boot datasource properties
3 static PostgreSQLContainer postgres = new PostgreSQLContainer("postgres:16-alpine")
4     .withDatabaseName("testdb")
5     .withInitScript("init-postgres-users.sql");
```

Same engine, same version, same dialect as production.

Keep Testcontainers Fast

- Enable container reuse where possible: `.withReuse(true)`
- Prefer a singleton container per test run over `@Testcontainers` (which starts one per test class)
- Speed up startup with prebuilt images that already contain your schema and seed data

```
1 private static PostgreSQLContainer postgres =  
2     new PostgreSQLContainer("myteampostgres:42");  
3  
4 static {  
5     postgres.start(); // started once, shared by every test that needs it  
6 }
```

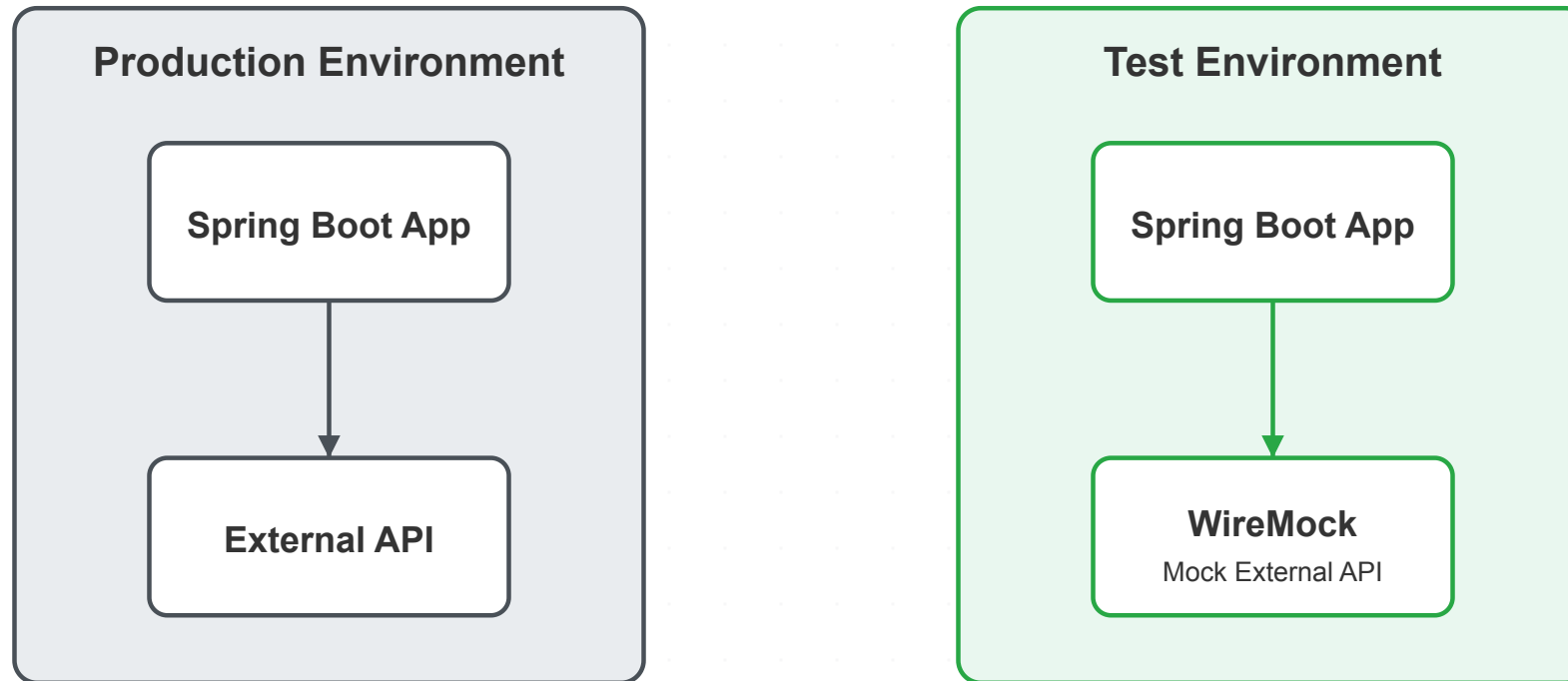
Must-Have #2: WireMock (Real HTTP)

Use case: your application calls a remote API. You want to exercise your own HTTP client, serialization and error handling - without depending on someone else's uptime.

Mocking the client with Mockito skips exactly the part that breaks in production:

- Wrong URL or query parameter
- Unexpected response shape
- Timeouts, 500s, and retry behaviour

WireMock: HTTP Service Stubbing for Testing



Key Benefits of WireMock

- Controlled test environment
- No network dependency
- Predictable responses
- Simulate error conditions
- Fast test execution
- Java integration

Be Aware of: Watermelon Tests

... green on the outside, red on the inside.

- **100% coverage** and still broken: coverage measures which lines *ran*, not which behavior was *verified*
- A test that asserts the mock works as previously instructed
- Auto-generated tests can give you the **feeling** of safety
- Agents produce these at scale: plausible names, green checks, zero judgment

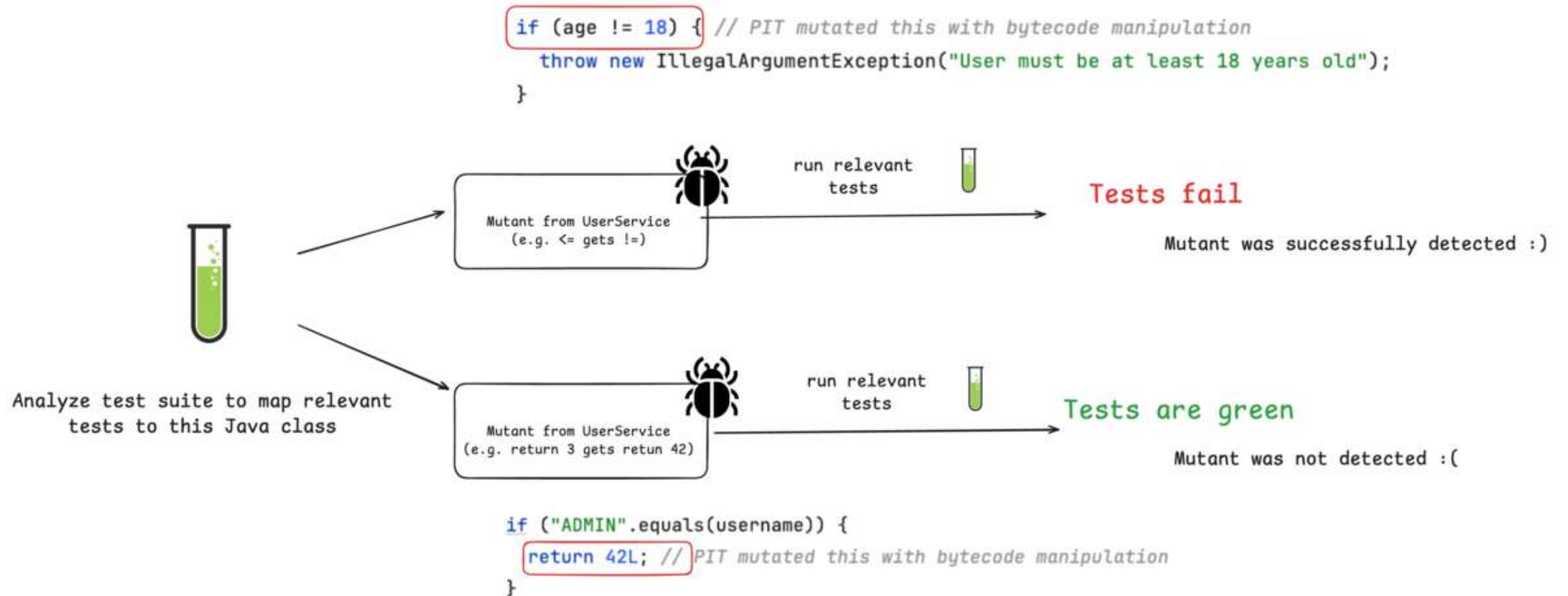
Let's Challenge Code Coverage

Imagine a set of unit tests for this isolated business logic:

```
1  public Long registerUser(int age, String username) {  
2  
3      if (age <= 18) {  
4          throw new IllegalArgumentException("User must be at least 18 years old");  
5      }  
6  
7      if ("ADMIN".equalsIgnoreCase(username)) {  
8          throw new IllegalArgumentException("Username 'ADMIN' is not allowed");  
9      }  
10  
11     // ...  
12  
13 }
```

Idea: Introduce Regressions to Verify Test Quality

... with the help of PIT:



The Three Myths, Busted

- 01** Most tests need no Spring context - climb only when forced to
- 02** Your suite is not slow, it is restarting contexts - cache, align, parallelize
- 03** Green is not proof - real infrastructure, real HTTP, meaningful assertions

Upcoming Talk at the DATEV Coding Festival

Ship Fast, Sleep Well: Fearless Spring Boot Deployments in the AI Era - my next talk at the **DATEV Coding Festival**.

We pick up where today stops: shipping the code an agent wrote, without losing sleep over it. Come and join:

Oct 6

2026

14:30

to 15:15

DATEV

Coding Festival

What I Do to Build Confidence in the Agentic Coding Era

- **Review generated tests** like a pull request from a new hire
- **Make the rules executable**: ArchUnit rules, testing conventions in `CLAUDE.md` / `AGENTS.md`, mutation score thresholds in CI
- **Custom skills**: define once per project how testing is tackled, with best practices and antipatterns
- **Keep the feedback loop tight**: fast, trustworthy tests let the agent iterate without me watching every step

My Agentic Testing Setup for Spring Boot

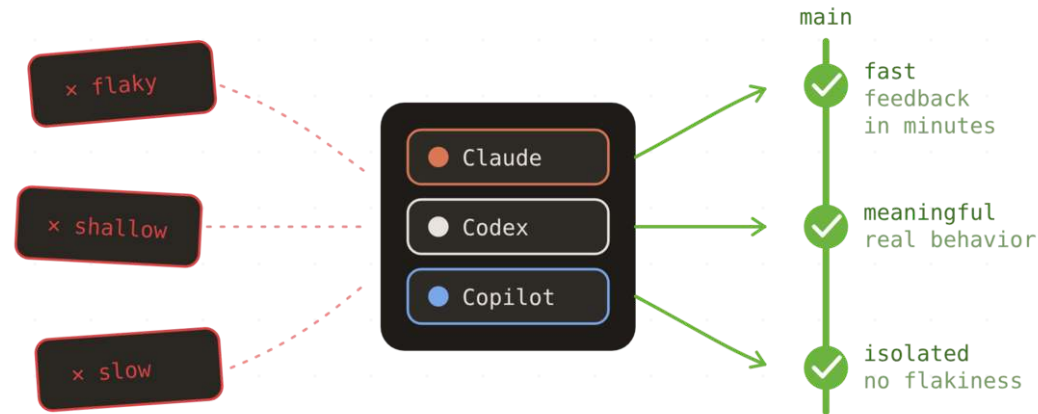
Define a **skillset** for fast and comprehensive tests, including a **test strategy** for the given project:

```
1  .claude/skills/  
2  |— unit-testing      fast tests without context bloat  
3  |— slice-testing     right-sized Spring context slices  
4  |— slice-testing-webmvc web layer with @WebMvcTest  
5  |— integration-testing full-context tests that stay fast  
6  |— e2e-testing       user journeys against the running app  
7  |— testcontainers-setup real infrastructure, reused containers  
8  |— test-setup-reviewer flags test anti-patterns for you
```

Each skill carries rules, references, best practices and antipatterns, described as code and text.

Get Notified

I am building an **agentic Spring Boot testing course**: how to make code agents produce tests you would have written yourself.



Agentic Testing for Spring Boot

From AI-generated noise to confidence in every commit

See the QR code on the slide for more information.

You can delegate the typing.

You can't delegate the ownership.

You get paged at 3 AM.

**Invest in a test suite that gives
you **confidence in every commit.****



Joyful Testing!

Get notified for Agentic Testing for Spring Boot:



- [LinkedIn](#) (Philip Riecks)
- [Mail](mailto:philip@pragmatech.digital) (philip@pragmatech.digital)

